



UNIVERSITY PROJECT TECHNICAL DOCUMENTATION

Circle Health

Complete Project, Requirements and Database Guide

DOCUMENT	System overview, requirements traceability, architecture, verification and PostgreSQL schema
VERSION	1.0 - Submission-ready application
DATE	20 August 2026
SYSTEM	React + Spring Boot + PostgreSQL full-stack health planning platform
DEMO	adhiraj@example.com / password123

Prepared as a formal implementation guide and requirements traceability record. All implementation statements and SQL structure were verified against the repository and the running development database.

Contents

The following sections describe the complete implemented system and provide traceability from stated requirements to code, APIs, persistence and verification evidence.

Contents	2
1. Executive Summary	4
1.1 Project objectives	4
1.2 Scope and boundaries	4
2. System Architecture	5
2.1 Frontend structure	5
2.2 Backend structure	5
3. Technology Stack	6
3.1 Protected frontend routes	6
4. Functional Requirements and Traceability	7
5. Non-Functional Requirements and Traceability	12
6. Principal User Workflows	15
6.1 Deterministic health calculations	15
7. REST API Catalogue	16
7.1 API response and validation policy	16
8. Database Design	17
8.1 Integrity rules	17
8.2 Verified demo data	18
9. Security, Privacy and Validation	19
10. Verification and Quality Assurance	19
10.1 Automated test coverage	20
11. Setup, Configuration and Operation	21
11.1 Database	21
11.2 Backend	21
11.3 Frontend	21
11.4 Environment variables	21
11.5 Production builds	21

12. Known Limitations and Future Production Considerations	21
13. Conclusion	22
Appendix A. Repository Traceability Map	23
Appendix B. Complete PostgreSQL Schema SQL	24

1. Executive Summary

Circle Health is a desktop-first fitness and nutrition planning application designed to consolidate routine health logging, analytics, goal tracking and personalised guidance into a cohesive experience. The product follows an Apple Health-inspired visual philosophy while remaining a conventional web application suitable for university demonstration and assessment.

The completed solution is a three-tier full-stack system. A React and TypeScript single-page application communicates with a Java Spring Boot REST API using JWT-authenticated JSON. Spring Data JPA persists user-owned records in PostgreSQL. OpenAI and Open Food Facts are optional enhancements: deterministic fallbacks and manual entry keep the complete demonstration functional without paid credentials or external availability.

The application deliberately uses one backend service rather than microservices. This design reduces operational complexity, keeps transactions and ownership checks straightforward, and is proportionate to the assessment scope. Shared services provide a single source of truth for nutrition targets, daily totals, health-calendar scoring, streaks, goals and conversational answers.

Current verification state: frontend lint and production build pass; the backend package succeeds; 59 automated backend tests pass with zero failures; the demo account authenticates successfully; and the live database contains coherent seeded records across every principal feature.

1.1 Project objectives

- Provide one reliable place to log meals, workouts, water, sleep and body measurements.
- Turn persisted records into understandable daily, weekly and monthly health context.
- Support planning through goals, workout plans, meal plans and consolidated groceries.
- Demonstrate responsible AI enhancement without making core workflows dependent on external credentials.
- Deliver a polished, responsive and assessment-friendly desktop web product with clear architecture.

1.2 Scope and boundaries

Circle Health provides general fitness and wellness planning. It is not a medical device, does not diagnose conditions and does not replace professional advice. Browser camera behavior depends on device permission and secure-context support. Password reset currently returns a privacy-preserving acknowledgement but does not deliver production email. Deployment credentials and production infrastructure are intentionally external to the repository.

2. System Architecture

The architecture is a layered monolith with a separately built browser client. Every user-owned request obtains identity from the authenticated Spring Security principal; resource IDs are resolved within that ownership boundary.

REACT CLIENT
Routes, pages, charts, forms, drawers, theme, typed API client
HTTPS / JSON REST / JWT
SPRING BOOT API
Controller -> DTO validation -> service -> repository -> entity
JPA / HIBERNATE
POSTGRESQL
Relational user data, plans, caches, reminders and notifications

Optional outbound integrations originate only from backend services. OpenAI receives bounded structured context and returns parsed JSON. Open Food Facts requests occur only after the barcode cache misses. Neither external service has direct access to the database.

2.1 Frontend structure

- pages - lazy-loaded route-level screens such as Dashboard, Log, Progress, Plans and Ask Circle.
- components - shared shell, cards, drawers, calendar, barcode scanner, exercise media, command palette and design-system primitives.
- services/api.ts - one typed HTTP boundary, base URL configuration, bearer token attachment and consistent error handling.
- context - authentication and light/dark/system theme state.
- types.ts - stable response contracts used across components.

2.2 Backend structure

- controller - REST route mapping and authenticated principal acquisition.
- dto - validation constraints and API-specific request/response models.
- service - calculations, transaction boundaries, ownership checks, aggregations and fallbacks.
- repository - Spring Data access methods scoped by user and date ranges.
- entity - relational mappings, uniqueness, cascading and orphan-removal behavior.
- security/config - JWT processing, password encoding, CORS and environment-driven configuration.
- exception - central conversion of expected failures into readable HTTP errors.

3. Technology Stack

Layer	Technology	Purpose in Circle Health
Frontend	React 19.2.8	Component UI and state-driven interaction
Frontend	TypeScript 6.0.2	Compile-time contracts and safer refactoring
Frontend	Vite 8.2.0	Development server and optimised production bundling
Frontend	Tailwind CSS 3.4.17	Responsive semantic design system
Frontend	React Router 7.18.2	Protected client-side routes and contextual navigation
Frontend	Recharts 3.10.1	Responsive health and trend visualisations
Frontend	Lucide React 1.31.0	Consistent accessible interface icons
Frontend	ZXing Browser 0.2.1	Browser-camera EAN/UPC decoding
Backend	Java 21	Primary application language and runtime target
Backend	Spring Boot 3.5.5	Web, configuration, validation and application lifecycle
Backend	Spring Security + JWT 0.12.6	Password authentication and signed bearer tokens
Backend	Spring Data JPA / Hibernate	Repositories, ORM and transaction management
Database	PostgreSQL	Relational persistence and integrity constraints
Database	PostgreSQL 17 Alpine (Compose)	Repeatable local database container
External	OpenAI Responses API	Optional structured plans, insights and chat enhancement
External	Open Food Facts	Barcode product and nutrition lookup
Testing	JUnit 5, Mockito, Spring Test	Deterministic backend verification
Quality	Oxlint + TypeScript compiler	Frontend static analysis and build verification

The Docker configuration targets PostgreSQL 17. The database used for the verified schema dump was PostgreSQL 14.20; the implemented DDL uses compatible standard PostgreSQL features.

3.1 Protected frontend routes

Public routes are **/login** and **/register**. The protected application shell exposes: **/dashboard**, **/log**, **/workouts**, **/workouts/new**, **/workouts/:id**, **/nutrition**, **/water**, **/sleep**, **/body**, **/progress**, **/goals**, **/plans**, **/insights**, **/settings**.

4. Functional Requirements and Traceability

A functional requirement describes observable system behavior. Each requirement below states what the application must do, how the completed implementation fulfils it, and the repository/API/database evidence used to verify that claim.

FR-01 | User registration, login and logout

REQUIREMENT	The system shall allow a user to create an account, authenticate using email and password, maintain an authenticated session and sign out.
HOW IT IS MET	React authentication screens call Spring Security endpoints. Passwords are stored as hashes and successful authentication returns a signed JWT used by the typed API client. Protected routes redirect unauthenticated users.
EVIDENCE	Routes /login and /register; POST /api/auth/register and /api/auth/login; AuthContext and JWT filter. Demo login adhiraj@example.com was verified with HTTP 200.

FR-02 | Profile and dietary setup

REQUIREMENT	The user shall maintain identity, age, gender, height, weight, activity level, fitness goal and detailed dietary preferences.
HOW IT IS MET	Settings provides Profile and Diet sections. UserProfile persists physiological targets plus pattern, restrictions, allergies, intolerances, disliked foods, cuisines, preferred proteins, preparation constraints and budget preference.
EVIDENCE	GET/PUT /api/profile; user_profile and profile_* collection tables; Jakarta validation and profile ownership through the authenticated principal.

FR-03 | Nutrition target and BMI calculation

REQUIREMENT	The system shall calculate BMI, BMR, TDEE, calorie and macronutrient targets from profile information and fitness goal.
HOW IT IS MET	NutritionCalculatorService applies deterministic formulas and activity/goal adjustments. The same persisted targets are consumed by Dashboard, Nutrition, Progress, Calendar and Ask Circle.
EVIDENCE	POST /api/nutrition/targets; Settings Targets view; NutritionCalculatorServiceTest passes. Targets are not independently hard-coded in React.

FR-04 | Nutrition and meal tracking

REQUIREMENT	Users shall create meals containing one or more foods, quantities and serving units, with daily calorie, protein, carbohydrate and fat totals.
HOW IT IS MET	The unified Log Meal workspace supports meal metadata, food search, multiple food rows, scaled quantities and running totals. Meal, MealFood and Food entities persist the relationship and nutrition basis.
EVIDENCE	GET/POST/DELETE /api/meals; meal, meal_food and food tables; Dashboard and Nutrition totals derive from the same NutritionTotalsCalculator.

FR-05 | Custom foods and reusable nutrition data

REQUIREMENT	Users shall create custom foods and reuse previously logged foods or meals.
HOW IT IS MET	Custom food creation is persisted with owner_id. Food shortcuts expose recent/frequent items and meal repeat endpoints recreate an owned meal safely.
EVIDENCE	POST /api/foods; GET /api/meals/food-shortcuts; POST /api/meals/{id}/repeat and /repeat-last; MealServiceOwnershipTest verifies isolation.

FR-06 | Meal quality scoring

REQUIREMENT	The application shall produce a transparent 0-100 meal quality score with explanatory signals.
HOW IT IS MET	A deterministic algorithm evaluates protein, fibre, macro balance, sugar, saturated fat and calorie density where data exists. Scores are stored on meals and presented as general guidance rather than medical authority.
EVIDENCE	quality_score column on meal; score appears in meal responses and Nutrition UI; missing optional nutrients are handled without failure.

FR-07 | Barcode food lookup

REQUIREMENT	Users shall scan or manually enter EAN/UPC barcodes and add resolved products to nutrition logs.
HOW IT IS MET	ZXing accesses the browser camera. The backend checks PostgreSQL first, requests Open Food Facts on a cache miss, normalizes solids, liquids and serving-based products, stores successful results, and returns readable failures.
EVIDENCE	GET /api/foods/barcode/{barcode}; FoodLookupServiceTest includes cache/fallback cases; Food stores measurement type, basis quantity/unit, package and suggested serving metadata.

FR-08 | Workout logging and history

REQUIREMENT	Users shall log workouts with multiple exercises, sets, repetitions, weight, duration and notes, then view or delete history.
HOW IT IS MET	The desktop workout builder uses an exercise list and sticky session summary. Workout owns ordered WorkoutExercise rows through cascade/orphan removal. History exposes duration, exercise count and volume.
EVIDENCE	GET/POST /api/workouts; GET/DELETE /api/workouts/{id}; workout and workout_exercise tables; live demo database contains 7 workouts and 27 workout exercises.

FR-09 | Exercise library and demonstrations

REQUIREMENT	Users shall search/filter exercises, inspect instructions and demonstration media, favourite entries and add an exercise to a workout.
HOW IT IS MET	The library supports query, muscle, equipment and difficulty filters. A consistent drawer displays setup/execution cues, optional privacy-enhanced YouTube embed, favourite action and workout-builder link.
EVIDENCE	GET /api/exercises; PUT/DELETE /api/exercises/{id}/favourite; exercise and favourite_exercise tables; 38 seeded exercises.

FR-10 | Water intake tracking

REQUIREMENT	Users shall add water using quick amounts or a custom amount and view progress against their target.
HOW IT IS MET	Quick Log, Log Water, Dashboard and command palette call the same persisted endpoint. Water summary returns daily totals and trend; latest entry can be undone.
EVIDENCE	POST /api/water; DELETE /api/water/latest; GET /api/water/summary; WaterLog validation rejects non-positive and excessive values.

FR-11 | Sleep logging

REQUIREMENT	Users shall create, edit, delete and review sleep entries containing bedtime, wake time, quality and notes.
HOW IT IS MET	SleepLog persists start/end timestamps. Service validation enforces chronological, plausible duration and quality rules. Summary calculates last night, weekly average, quality and chart history using the wake date.
EVIDENCE	GET/POST /api/sleep; PUT/DELETE /api/sleep/{id}; GET /api/sleep/summary; HealthServiceTest covers sleep behavior.

FR-12 | Body measurement tracking

REQUIREMENT	Users shall record optional weight, body fat, chest, waist, hips, arms, thighs and notes by date.
HOW IT IS MET	The Body screen and unified logger support partial records, history management and trends. The service requires at least one meaningful measurement and validates plausible ranges.
EVIDENCE	GET/POST /api/measurements; PUT/DELETE /api/measurements/{id}; summary endpoint and body_measurement table; 5 coherent seeded entries.

FR-13 | Unified health logging

REQUIREMENT	Routine workout, meal, water, sleep and body entry shall be available from one route without losing the specialist history pages.
HOW IT IS MET	The /log workspace uses category selection, quick actions and context-specific sticky summaries. Each form reuses existing persistence APIs, shows success/error states and remains on the route after save.
EVIDENCE	Protected /log route; Quick Log drawer and top-bar action; forms call workouts, meals, water, sleep and measurements APIs.

FR-14 | Dashboard and daily overview

REQUIREMENT	The user shall receive a current, coherent view of calories, macros, hydration, sleep, activity, weight, streak, health score and immediate actions.
HOW IT IS MET	DashboardService and TodayService aggregate persisted records. The compact metric strip, daily score, activity chart, timeline, pinned metrics, goals and contextual actions consume those shared summaries.
EVIDENCE	GET /api/dashboard and /api/today; no client-side duplicate target calculations; smart links route to relevant log/detail pages.

FR-15 | Progress analytics

REQUIREMENT	The application shall provide weekly and monthly analytics across workouts, nutrition, hydration, sleep and body measurements.
HOW IT IS MET	ProgressService performs server-side aggregation, averages, training volume, target-hit counts, comparison and recovery calculations. Recharts renders sparse and populated datasets responsively.
EVIDENCE	GET /api/progress?range=week month, /compare and /recovery; ProgressServiceTest passes.

FR-16 | Health adherence calendar and streak

REQUIREMENT	Users shall inspect monthly ON_TRACK, PARTIAL, OFF_TRACK and NO_DATA states, day explanations and current/longest logging streaks.
HOW IT IS MET	HealthCalendarService scores only applicable categories, keeps unscheduled workout absence neutral and treats body measurements as informational. StreakService derives consecutive dates from persisted meaningful activities.
EVIDENCE	GET /api/progress/calendar and /api/streak; HealthCalendarServiceTest has 11 cases covering thresholds, boundaries, ownership and no-data behavior.

FR-17 | Goals and timelines

REQUIREMENT	Users shall create, edit, pause, resume, archive and delete measurable goals with progress, status, pace and projection.
HOW IT IS MET	Goal metadata is persisted while current values are derived from existing health records to avoid duplicate measurements. Direction-aware mathematics supports weight, body fat, waist, nutrition, water, sleep, frequency and streak goals.
EVIDENCE	GET/POST/PUT/DELETE /api/goals plus lifecycle endpoints; goals table and indexes; GoalMathTest and GoalServiceTest provide 20 passing cases.

FR-18 | Ask Circle conversational guidance

REQUIREMENT	The user shall ask detailed health-data questions in a simple conversational interface and open relevant application pages from responses.
HOW IT IS MET	Ask Circle maintains a session message thread, evidence-backed answers, suggested prompts, meal and finish-day responses, direct hydration actions and contextual links. OpenAI can enhance wording; deterministic analysis remains available without credentials.
EVIDENCE	POST /api/ai/ask; GET /api/ai/meal-suggestions and /finish-day; live smoke tests returned specific week, recovery, protein, goal and hydration evidence with valid routes.

FR-19 | Personalised AI insights

REQUIREMENT	The application shall analyse profile, workouts, nutrition, water, sleep and measurements and provide concise non-medical recommendations.
HOW IT IS MET	InsightService assembles a bounded recent-data context and AiService requests structured JSON when configured. A deterministic fallback produces wins, attention areas and recommendations.
EVIDENCE	POST /api/ai/insights; AiServiceFallbackTest verifies credential-free behavior and structured responses.

FR-20 | Workout plan generation

REQUIREMENT	Users shall generate, save, reopen and delete structured workout plans from goal, experience, frequency, duration, equipment and preferences.
HOW IT IS MET	AiService returns typed plan JSON and PlanService persists it. The fallback selects compatible exercises/equipment and produces repeatable day structures when OpenAI is unavailable.
EVIDENCE	POST /api/plans/workouts/generate; GET/DELETE /api/plans/workouts/{id}; generated_workout_plan table.

FR-21 | Meal plan generation and dietary compatibility

REQUIREMENT	Users shall generate and persist seven-day meal plans using targets, dietary pattern, allergies, exclusions, dislikes and meal frequency.
HOW IT IS MET	Settings profile values pre-fill generation. Structured AI output is parsed and checked against DietRules; deterministic fallback respects the same profile. The UI uses compact weekly accordions.
EVIDENCE	POST /api/plans/meals/generate; GET/DELETE /api/plans/meals/{id}; generated_meal_plan table; DietaryProfileServiceTest and DietCompatibilityServiceTest pass.

FR-22 | Grocery list management

REQUIREMENT	The system shall aggregate meal-plan ingredients and allow check, uncheck, custom item, quantity editing, deletion, clearing, text export and print.
HOW IT IS MET	PlanService normalizes repeated ingredients into relational GroceryList/GroceryItem data. All item state persists, while the browser provides export and printable presentation.
EVIDENCE	POST /api/grocery-lists/from-meal-plan/{id}; list/item CRUD endpoints; grocery_lists and grocery_item tables with ownership constraints.

FR-23 | Reminders and notifications

REQUIREMENT	Users shall configure recurring in-app reminders and manage unread notifications.
HOW IT IS MET	Reminder rows persist type, enabled state, time and recurrence. NotificationController materializes due reminders, returns unread count and supports individual/all read actions.
EVIDENCE	GET/PUT /api/reminders; GET /api/notifications; PUT /read and /read-all; reminder and notification tables.

FR-24 | Search, navigation and contextual actions

REQUIREMENT	The user shall navigate the complete product efficiently and invoke common actions from context.
HOW IT IS MET	A persistent responsive sidebar, global Cmd/Ctrl+K command palette, route-aware active state, Quick Log drawer, detail drawers and answer action links connect the product without duplicate feature implementations.
EVIDENCE	React Router route map; AppShell, CommandPalette and DetailDrawer shared components; all protected routes remain directly addressable.

FR-25 | Personal data export and appearance

REQUIREMENT	Users shall export owned health data and choose light, dark or system appearance.
HOW IT IS MET	Profile export returns CSV sections for profile, workouts, nutrition, water, sleep, measurements and goals. ThemeContext applies saved/system mode before React renders; top-bar and Settings controls share the same preference.
EVIDENCE	GET /api/profile/export; Circle Health CSV filenames; pulse_theme local preference; frontend lint/build verification.

5. Non-Functional Requirements and Traceability

Non-functional requirements describe the quality attributes and constraints under which the features operate. They are phrased as assessable system qualities rather than additional product features.

NFR-01 | Usability and information hierarchy

REQUIREMENT	The desktop application should be understandable without training and minimise routine-entry friction.
HOW IT IS MET	Large headings, compact metric strips, consistent semantic colours, unified logging, sticky summaries, contextual actions, polished empty states and conversational guidance reduce navigation and cognitive load.
EVIDENCE	Shared UI primitives and semantic tokens; common actions are available from Dashboard, /log, top bar and command palette.

NFR-02 | Responsive behavior

REQUIREMENT	Primary workflows shall remain usable from large desktop screens to tablet and narrow mobile browser widths.
HOW IT IS MET	Tailwind responsive grids collapse dual-column builders, sticky summaries stack below editors, drawers become compact sheets and charts use responsive containers.
EVIDENCE	Design targets include 1440, 1280, 1024, 768 and approximately 390 px widths; production CSS is generated from responsive classes.

NFR-03 | Accessibility

REQUIREMENT	Interactive controls shall expose meaningful labels and state beyond colour alone.
HOW IT IS MET	Buttons use aria-labels, the theme control uses role=switch and aria-checked, navigation links expose labels/tooltips, calendar states include text/icons and forms use visible labels.
EVIDENCE	Semantic HTML, keyboard command access, focus-visible outlines, readable contrast tokens and non-autoplay video embeds.

NFR-04 | Performance

REQUIREMENT	The application should load and interact promptly for university-scale data volumes.
HOW IT IS MET	Route pages are lazy-loaded, Recharts datasets are bounded, barcode scanning is isolated in a separate bundle, API responses use targeted DTOs and indexed goal queries avoid unnecessary payloads.
EVIDENCE	Vite production build succeeds; generated bundles are code-split by feature page.

NFR-05 | Reliability and graceful degradation

REQUIREMENT	Core workflows shall remain demonstrable when optional external APIs fail or credentials are absent.
HOW IT IS MET	OpenAI calls are wrapped with validated structured parsing and deterministic fallbacks. Open Food Facts errors become readable responses while manual/custom food entry remains available.
EVIDENCE	AIServiceFallbackTest and FoodLookupServiceTest pass; no paid credential is required for the demo.

Non-Functional Requirements - Continued

NFR-06 | Data integrity

REQUIREMENT	Persisted relationships and calculations shall remain coherent across create, update, delete and refresh.
HOW IT IS MET	JPA relationships use foreign keys, unique constraints, cascading and orphan removal where ownership requires it. Shared services centralise nutrition totals, targets and analytics.
EVIDENCE	Verified PostgreSQL DDL; unique email/barcode/exercise/favourite constraints; HealthCalendar and ownership tests.

NFR-07 | Security and ownership

REQUIREMENT	Authenticated users shall not access another user's health records by changing resource identifiers.
HOW IT IS MET	Controllers derive identity from Spring Authentication rather than accepting user IDs. Services/repositories resolve owned entities using authenticated email/user ID. Passwords are hashed and APIs require JWT except auth routes.
EVIDENCE	Spring Security filter chain, JWT service, ownership-aware service methods and MealServiceOwnershipTest.

NFR-08 | Privacy

REQUIREMENT	Health data and secrets shall be handled with appropriate separation and minimal exposure.
HOW IT IS MET	DTOs prevent accidental entity serialization, exports are scoped to the current user, environment variables hold secrets and AI prompts receive bounded structured health context with a non-medical disclaimer.
EVIDENCE	No secrets in repository; .env examples document configuration; schema export contains structure only.

NFR-09 | Maintainability

REQUIREMENT	The codebase shall remain understandable and modifiable by a small project team.
HOW IT IS MET	The backend uses controller/service/repository/entity/dto/security/config layers in one monolith. The frontend separates pages, shared components, typed API access, contexts and response contracts.
EVIDENCE	Single Spring Boot application, no microservices, reusable Card/Drawer/Theme/Brand components and central error handling.

NFR-10 | Testability

REQUIREMENT	Critical deterministic behavior shall be verifiable without external services.
HOW IT IS MET	Business formulas and fallbacks are implemented in services with unit tests using controlled repositories/mocks. Frontend type checking and linting detect build/runtime-contract errors.
EVIDENCE	59 backend tests pass; npm run lint and npm run build pass; Maven package succeeds.

Non-Functional Requirements - Continued

NFR-11 | Portability and deployment

REQUIREMENT	The solution shall run locally and support environment-driven deployment.
HOW IT IS MET	Vite uses VITE_API_URL; Spring uses database, CORS, JWT, OpenAI and port variables. Docker Compose provides PostgreSQL and Maven/npm wrappers standardise builds.
EVIDENCE	docker-compose.yml, .env.example files, Spring prod profile and documented setup/build commands.

NFR-12 | Interoperability

REQUIREMENT	External integrations and client-server communication shall use standard protocols and stable contracts.
HOW IT IS MET	Frontend/backend communicate through JSON REST over HTTP with bearer JWT. OpenAI uses the Responses API, Open Food Facts supplies barcode data and CSV/plain text support exports.
EVIDENCE	Typed frontend API service, DTO records and normalized external response handling.

NFR-13 | Error handling and validation

REQUIREMENT	Invalid user input shall produce understandable errors rather than uncaught failures.
HOW IT IS MET	Frontend fields apply limits and required states; Jakarta Bean Validation is the backend authority; GlobalExceptionHandler returns consistent errors; network actions show loading/error states.
EVIDENCE	Validation annotations cover plan/profile/log DTOs; health and food tests include boundary/error cases.

NFR-14 | Consistency and auditability

REQUIREMENT	Metrics shown in different pages shall use the same source of truth and transparent algorithms.
HOW IT IS MET	Dashboard, Today, Progress, Calendar, Goals and Ask Circle reuse persisted records and shared services. Date-only semantics and wake-date sleep assignment prevent cross-page drift.
EVIDENCE	Calendar tests cover thresholds/month boundaries; documentation states formulas and health-score categories; API smoke results matched persisted demo data.

6. Principal User Workflows

Workflow	End-to-end behavior
Daily logging	Sign in -> open /log or Quick Log -> choose Meal/Workout/Water/Sleep/Body -> validate -> save -> remain in context -> Dashboard/Progress immediately read the same persisted record.
Nutrition barcode	Open Meal log -> scan or enter code -> database cache lookup -> Open Food Facts on miss -> normalize basis/unit/package -> adjust amount -> scale macros -> add to meal -> persist.
Progress review	Open Progress -> choose week/month -> inspect server aggregation -> move calendar month -> select day -> review category scores, wins, attention areas and linked actions.
Goal tracking	Create goal -> service captures starting context -> current value derives from logs -> dashboard/Goals displays progress and status -> pause/resume/archive as required.
Plan to groceries	Generate structured meal plan -> validate dietary compatibility -> persist -> expand days -> generate grocery list -> aggregate ingredients -> manage checked/custom items -> export or print.
Ask Circle	Enter detailed question -> backend assembles current/weekly/goal context -> optional OpenAI structured answer or deterministic fallback -> render evidence -> navigate or perform permitted action.

6.1 Deterministic health calculations

Calculation	Rule
BMI	weight (kg) / height (m) ²
BMR	Mifflin-St Jeor equation using profile age, sex, height and weight
TDEE	BMR multiplied by activity-level factor
Calorie target	TDEE adjusted conservatively for lose, maintain or muscle-gain goal
Macros	Protein target based on body weight; remaining calories distributed across carbohydrate and fat
Meal quality	Bounded 0-100 heuristic using available protein, fibre, macro balance, sugar, saturated fat and density
Hydration day	>=90% target ON_TRACK; 60-89% PARTIAL; <60% OFF_TRACK when data exists
Sleep day	Duration and quality contribute only when a sleep entry exists; date uses wake date
Activity day	Logged workout contributes positively; no unscheduled workout remains neutral
Overall day	Average of meaningful applicable categories; body measurements never reduce the score
Streak	Consecutive dates containing at least one meaningful meal, workout, water or sleep activity
Goal projection	Recent dated trend projected only when sufficient data moves toward the target

7. REST API Catalogue

All routes below use the /api prefix. Except registration, login and the acknowledgement-only password reset route, requests require a valid bearer JWT. User-owned endpoints derive identity from the token rather than a client-provided user ID.

Domain	Methods	Representative paths	Responsibility
Authentication	POST	/api/auth/register, /login	Create account and issue JWT
Profile	GET/PUT	/api/profile	Read/update profile and calculated targets
Export	GET	/api/profile/export	Download authenticated user's CSV data
Dashboard	GET	/api/dashboard, /api/today	Daily metrics, timeline and highlights
Workouts	GET/POST/DELETE	/api/workouts/[id]	History, create, detail, repeat and delete
Exercises	GET/PUT/DELETE	/api/exercises	Search/filter library and favourites
Meals	GET/POST/DELETE	/api/meals	History, logging, shortcuts and repeat
Foods	GET/POST	/api/foods, /barcode/{code}	Search, custom foods and barcode cache
Water	POST/DELETE/GET	/api/water, /water/summary	Quick-add, undo and trends
Sleep	GET/POST/PUT/DELETE	/api/sleep/[id], /summary	Full sleep CRUD and analytics
Measurements	GET/POST/PUT/DELETE	/api/measurements/[id], /summary	Body CRUD and trends
Progress	GET	/api/progress, /compare, /recovery	Cross-domain aggregation
Calendar/streak	GET	/api/progress/calendar, /api/streak	Daily adherence and consecutive activity
Goals	GET/POST/PUT/DELETE	/api/goals/[id]	CRUD plus pause, resume and archive
AI guidance	POST/GET	/api/ai/*	Insights, Ask Circle, meal guidance and plans
Saved plans	GET/POST/DELETE	/api/plans/workouts, /plans/meals	Generate and manage plans
Groceries	GET/POST/PUT/DELETE	/api/grocery-lists, /grocery-items	Generate and manage relational lists
Reminders	GET/PUT	/api/reminders/{type}	Persist schedule and enabled state
Notifications	GET/PUT	/api/notifications	Unread list and read actions

7.1 API response and validation policy

- Controllers accept validated DTOs and do not return JPA entities directly.
- Expected client mistakes become readable 4xx responses through central exception handling.
- Create operations generally return HTTP 201; successful deletes return HTTP 204.
- AI outputs are parsed into fixed structures; malformed or unavailable external output falls back safely.
- Date range aggregation is performed in services, not duplicated independently across React components.

8. Database Design

The schema is relational and centred on `app_users`. Most health data is a time-series child of a user. Join entities preserve quantities and ordering for meals and workouts. Generated plan bodies are stored as structured JSON text because their nested shape is read as a unit, while interactive grocery state is relational.

Table/group	Purpose	Relationship
<code>app_users</code>	Account identity and password hash	Parent for user-owned records
<code>user_profile</code>	Physiology, targets and diet settings	One-to-one with <code>app_users</code>
<code>profile_* collections</code>	Allergies, exclusions, cuisine and protein sets	Many values per <code>user_profile</code>
<code>workout</code>	Session name, date, duration and notes	Many-to-one user
<code>workout_exercise</code>	Ordered exercise performance	Many-to-one workout and exercise
<code>exercise</code>	Shared seeded exercise catalogue	Referenced by workouts/favourites
<code>favourite_exercise</code>	Persistent per-user favourite	Unique user + exercise
<code>meal</code>	Meal event and quality score	Many-to-one user; owns <code>meal_food</code>
<code>meal_food</code>	Food quantity within a meal	Many-to-one meal and food
<code>food</code>	Nutrition basis, package and barcode metadata	Global/cache or owner-specific
<code>water_log</code>	Timestamped water quantity	Many-to-one user
<code>sleep_log</code>	Bed/wake timestamps, quality and notes	Many-to-one user
<code>body_measurement</code>	Dated optional anthropometric values	Many-to-one user
<code>goals</code>	Target metadata and lifecycle status	Current value derived from logs
<code>generated_* plan</code>	Structured saved workout/meal plan JSON	Many-to-one user
<code>grocery_lists/items</code>	Relational shopping state	Meal plan optional parent
<code>reminder/notification</code>	Scheduling and in-app delivery state	Many-to-one user

8.1 Integrity rules

- Primary keys use PostgreSQL identity-backed bigint values.
- `app_users.email`, `exercise.name` and `food.barcode` are unique.
- `user_profile.user_id` is unique, enforcing one profile per account.
- `favourite_exercise` enforces a unique user/exercise pair.
- Goal type, status and direction are constrained to defined enumerations and indexed by user/status and user/target date.
- Foreign keys link all user-owned and join records; JPA cascade/orphan rules protect normal UI delete workflows.
- `created_at` and `updated_at` audit timestamps originate from `BaseEntity`.

8.2 Verified demo data

1 Users	38 Exercises	7 Workouts
27 Workout exercises	33 Meals	7 Foods
9 Water logs	10 Sleep logs	5 Body measurements
4 Goals	2 Meal plans	1 Workout plans
2 Grocery lists	5 Reminders	6 Notifications

9. Security, Privacy and Validation

Control	Implementation
Authentication	Email/password login; BCrypt-compatible password encoding; signed JWT returned after successful authentication.
Authorization	Protected routes require JWT. The authenticated email is used to resolve user-owned resources.
Ownership	Workout, meal, food, health log, goal, plan, grocery, reminder and notification operations constrain queries to the current user.
Input validation	Bean Validation and explicit service rules enforce required fields, sizes, numeric ranges and chronological consistency.
Error safety	Central error conversion avoids exposing stack traces or raw persistence exceptions to normal clients.
External AI	Bounded structured data, strict JSON response parsing, non-medical instruction and deterministic fallback.
Food metadata	Incomplete allergen/ingredient metadata is disclosed; the UI does not claim allergen safety.
Secrets	Database credentials, JWT secret and API key are environment-driven; only development defaults/examples are documented.
Export	CSV data is generated for the authenticated user only.

Password-reset production email delivery is intentionally outside the current project scope. The endpoint returns the same acknowledgement whether or not an account exists, preventing simple account enumeration while preserving the visible workflow.

10. Verification and Quality Assurance

The following results were obtained from the repository on 20 August 2026 after the latest Circle Health and Ask Circle updates.

Check	Command/evidence	Result
Frontend lint	npm run lint	PASS
Frontend production build	npm run build	PASS
Backend automated tests	./mvnw test / package	59 passed; 0 failed
Backend package	Spring Boot executable JAR	PASS
Demo authentication	POST /api/auth/login	HTTP 200
Ask Circle smoke tests	Week, goals, protein, hydration and recovery questions	PASS with grounded evidence/actions
Database schema	pg_dump --schema-only from live PostgreSQL	Captured in Appendix B

10.1 Automated test coverage

Test class	Cases	Primary scope
GoalMathTest	10	Direction-aware progress and projection mathematics
GoalServiceTest	10	Goal lifecycle, ownership and derived values
HealthCalendarServiceTest	11	Daily statuses, thresholds, no-data, month boundaries and ownership
AIServiceFallbackTest	8	Structured credential-free plans and dietary constraints
FoodLookupServiceTest	6	Barcode normalization, cache and fallback behavior
HealthServiceTest	5	Water, sleep and measurement service behavior
DietaryProfileServiceTest	3	Diet profile normalization and persistence
DietCompatibilityServiceTest	3	Compatibility and incomplete-metadata handling
NutritionCalculatorServiceTest	1	Target calculation
ProgressServiceTest	1	Cross-domain progress aggregation
MealServiceOwnershipTest	1	User-owned nutrition isolation

11. Setup, Configuration and Operation

11.1 Database

```
docker compose up -d postgres
```

The default development database is pulse, with the development-only username and password pulse. Override all database values in non-local environments.

11.2 Backend

```
cd backend
./mvnw spring-boot:run
```

Requires Java 21 or newer. The API is available at <http://localhost:8080/api>.

11.3 Frontend

```
cd frontend
npm install
npm run dev
```

The development client is available at <http://localhost:5173>.

11.4 Environment variables

Variable	Purpose	Development behavior
DATABASE_URL	PostgreSQL JDBC URL	jdbc:postgresql://localhost:5432/pulse
DATABASE_USERNAME	Database account	pulse
DATABASE_PASSWORD	Database password	pulse (development only)
JWT_SECRET	Token signing secret	Development fallback; replace in production
OPENAI_API_KEY	Optional live AI	Empty uses deterministic fallback
OPENAI_MODEL	OpenAI model	gpt-4.1-mini
CORS_ORIGIN	Allowed frontend origin	http://localhost:5173
PORT	Backend port	8080
VITE_API_URL	Frontend API base URL	http://localhost:8080/api

11.5 Production builds

```
cd frontend
npm run lint
npm run build

cd ../backend
./mvnw test
./mvnw package
```

12. Known Limitations and Future Production Considerations

- Production password-reset email delivery is not implemented; the acknowledgement flow is present and intentionally does not disclose account existence.
- OpenAI-enhanced wording and generation require a valid key, network access and service availability. Deterministic alternatives remain fully usable.

- Camera scanning depends on browser permission, compatible hardware and secure-context rules; manual barcode entry remains available.
- Open Food Facts is community-maintained and may provide incomplete package, ingredient or allergen metadata. The system exposes uncertainty and allows manual correction.
- The development profile uses Hibernate schema update. A production deployment should replace this with versioned database migrations such as Flyway or Liquibase.
- Chat messages are maintained for the active browser session and are not persisted as health records.
- In-app reminders are evaluated when notifications are requested; no separate always-on push-delivery infrastructure is included.
- The application is educational wellness software and has not undergone medical-device validation or a formal security penetration test.

13. Conclusion

Circle Health satisfies the stated university-project objective as a polished, functional and demonstrable full-stack fitness and nutrition planner. The application supports complete persisted workflows, coherent cross-domain analytics, defensible deterministic calculations and optional AI enhancement without introducing unnecessary infrastructure.

The strongest architectural characteristic is consistency: day totals, targets, calendar status, streak, goals and Ask Circle draw from the same authenticated records and shared backend services. This allows a marker to create or modify data, refresh the application and observe the corresponding effect throughout the system.

Appendix A. Repository Traceability Map

Concern	Primary source
Frontend routes	frontend/src/App.tsx
Persistent shell/navigation	frontend/src/components/AppShell.tsx
Unified logging	frontend/src/pages/LogPage.tsx
Dashboard	frontend/src/pages/Dashboard.tsx
Progress/calendar	frontend/src/pages/ProgressPage.tsx and components/HealthCalendar.tsx
Ask Circle	frontend/src/pages/InsightsPage.tsx
Typed API	frontend/src/services/api.ts and frontend/src/types.ts
REST controllers	backend/src/main/java/com/pulse/controller
Business services	backend/src/main/java/com/pulse/service
Persistence entities	backend/src/main/java/com/pulse/entity
Repositories	backend/src/main/java/com/pulse/repository
Security	backend/src/main/java/com/pulse/security and config
Tests	backend/src/test/java/com/pulse/service
Runtime configuration	backend/src/main/resources/application.yml and docker-compose.yml
Exact SQL	docs/database-schema.sql

Appendix B. Complete PostgreSQL Schema SQL

The following DDL was generated from the verified running development database using `pg_dump --schema-only` with ownership and privilege statements removed. The same code is supplied separately at `docs/database-schema.sql` for direct execution and review.

```
--
-- Circle Health PostgreSQL schema
-- Generated from the verified local application database on 2026-08-20.
-- This file contains structure only; it does not contain user records or secrets.
--

-- Dumped from database version 14.20 (Homebrew)
-- Dumped by pg_dump version 14.20 (Homebrew)

SET statement_timeout = 0;
SET lock_timeout = 0;
SET idle_in_transaction_session_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SELECT pg_catalog.set_config('search_path', '', false);
SET check_function_bodies = false;
SET xmloption = content;
SET client_min_messages = warning;
SET row_security = off;

SET default_tablespace = '';

SET default_table_access_method = heap;

--
-- Name: app_users; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.app_users (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    email character varying(180) NOT NULL,
    name character varying(100) NOT NULL,
    password_hash character varying(255) NOT NULL
);

--
-- Name: app_users_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.app_users ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.app_users_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: body_measurement; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.body_measurement (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    arms_cm double precision,
    body_fat_percentage double precision,
    chest_cm double precision,
    hips_cm double precision,
    measured_on date NOT NULL,
    thighs_cm double precision,
    waist_cm double precision,
    weight_kg double precision,
    user_id bigint NOT NULL,
    notes character varying(1000)
);

--
-- Name: body_measurement_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.body_measurement ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.body_measurement_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);
```



```
-- Name: exercise; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.exercise (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    difficulty character varying(255),
    equipment character varying(255),
    instructions character varying(3000),
    media_url character varying(255),
    muscle_group character varying(255),
    name character varying(255) NOT NULL
);

--
-- Name: exercise_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.exercise ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.exercise_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: favourite_exercise; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.favourite_exercise (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    exercise_id bigint NOT NULL,
    user_id bigint NOT NULL
);

--
-- Name: favourite_exercise_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.favourite_exercise ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.favourite_exercise_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: food; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.food (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    barcode character varying(255),
    brand character varying(255),
    calories double precision,
    carbohydrates double precision,
    custom_food boolean NOT NULL,
    fat double precision,
    fibre double precision,
    name character varying(255) NOT NULL,
    protein double precision,
    serving_size double precision,
    serving_unit character varying(255),
    owner_id bigint,
    saturated_fat double precision,
    sugar double precision,
    allergen_tags character varying(2000),
    dietary_tags character varying(2000),
    ingredients_text character varying(5000),
    measurement_type character varying(255),
    nutrition_basis_quantity double precision,
    nutrition_basis_unit character varying(255),
    package_quantity double precision,
    package_unit character varying(255),
    suggested_serving_quantity double precision,
    suggested_serving_unit character varying(255)
);

--
-- Name: food_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.food ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.food_id_seq
    START WITH 1
    INCREMENT BY 1
```

```

        NO MINVALUE
        NO MAXVALUE
        CACHE 1
    );

--
-- Name: generated_meal_plan; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.generated_meal_plan (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    calorie_target integer,
    grocery_list_json text,
    name character varying(255) NOT NULL,
    plan_json text,
    saved boolean NOT NULL,
    user_id bigint NOT NULL
);

--
-- Name: generated_meal_plan_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.generated_meal_plan ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.generated_meal_plan_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: generated_workout_plan; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.generated_workout_plan (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    days_per_week integer,
    goal character varying(255),
    name character varying(255) NOT NULL,
    plan_json text,
    saved boolean NOT NULL,
    user_id bigint NOT NULL
);

--
-- Name: generated_workout_plan_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.generated_workout_plan ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.generated_workout_plan_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: goals; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.goals (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    completed_date date,
    direction character varying(20) NOT NULL,
    start_date date NOT NULL,
    start_value double precision,
    status character varying(20) NOT NULL,
    target_date date,
    target_value double precision NOT NULL,
    title character varying(140) NOT NULL,
    type character varying(40) NOT NULL,
    unit character varying(20) NOT NULL,
    user_id bigint NOT NULL,
    CONSTRAINT goals_direction_check CHECK (((direction)::text = ANY ((ARRAY['DECREASE'::character varying, 'INCREASE'::character varying, 'MAINTAIN'::character varying, 'AT_LEAST'::character varying, 'AT_MOST'::character varying])::text[]))),
    CONSTRAINT goals_status_check CHECK (((status)::text = ANY ((ARRAY['ACTIVE'::character varying, 'COMPLETED'::character varying, 'PAUSED'::character varying, 'FAILED'::character varying, 'ARCHIVED'::character varying])::text[]))),
    CONSTRAINT goals_type_check CHECK (((type)::text = ANY ((ARRAY['WEIGHT'::character varying, 'BODY_FAT'::character varying, 'WAIST'::character varying, 'PROTEIN'::character varying, 'CALORIES'::character varying, 'WATER'::character varying, 'SLEEP'::character varying, 'WORKOUT_FREQUENCY'::character varying, 'STREAK'::character varying])::text[])))
);

```

```
-- Name: goals_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.goals ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.goals_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: grocery_item; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.grocery_item (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    category character varying(255),
    checked boolean NOT NULL,
    name character varying(255) NOT NULL,
    quantity double precision,
    unit character varying(255),
    grocery_list_id bigint NOT NULL,
    item_order integer
);

--
-- Name: grocery_item_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.grocery_item ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.grocery_item_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: grocery_lists; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.grocery_lists (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    name character varying(255) NOT NULL,
    meal_plan_id bigint,
    user_id bigint NOT NULL
);

--
-- Name: grocery_lists_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.grocery_lists ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.grocery_lists_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: meal; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.meal (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    eaten_at timestamp(6) without time zone,
    meal_type character varying(255),
    name character varying(255) NOT NULL,
    quality_score integer,
    reusable boolean NOT NULL,
    user_id bigint NOT NULL
);

--
-- Name: meal_food; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.meal_food (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    quantity double precision,
    unit character varying(255),
```

```

        food_id bigint NOT NULL,
        meal_id bigint NOT NULL
    );

--
-- Name: meal_food_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.meal_food ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.meal_food_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: meal_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.meal ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.meal_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: notification; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.notification (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    message character varying(1000) NOT NULL,
    read_at timestamp(6) with time zone,
    title character varying(255) NOT NULL,
    type character varying(255) NOT NULL,
    user_id bigint NOT NULL
);

--
-- Name: notification_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.notification ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.notification_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: profile_cultural_preferences; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_cultural_preferences (
    profile_id bigint NOT NULL,
    preference character varying(255)
);

--
-- Name: profile_custom_allergies; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_custom_allergies (
    profile_id bigint NOT NULL,
    allergy character varying(255)
);

--
-- Name: profile_custom_cultural_preferences; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_custom_cultural_preferences (
    profile_id bigint NOT NULL,
    preference character varying(255)
);

--
-- Name: profile_custom_exclusions; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_custom_exclusions (
    profile_id bigint NOT NULL,
    exclusion character varying(255)
);

```

```
);

--
-- Name: profile_custom_intolerances; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_custom_intolerances (
    profile_id bigint NOT NULL,
    intolerance character varying(255)
);

--
-- Name: profile_custom_proteins; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_custom_proteins (
    profile_id bigint NOT NULL,
    protein_source character varying(255)
);

--
-- Name: profile_dietary_restrictions; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_dietary_restrictions (
    profile_id bigint NOT NULL,
    restriction character varying(255)
);

--
-- Name: profile_disliked_foods; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_disliked_foods (
    profile_id bigint NOT NULL,
    food character varying(255)
);

--
-- Name: profile_favourite_foods; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_favourite_foods (
    profile_id bigint NOT NULL,
    food character varying(255)
);

--
-- Name: profile_food_allergies; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_food_allergies (
    profile_id bigint NOT NULL,
    allergy character varying(255)
);

--
-- Name: profile_food_intolerances; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_food_intolerances (
    profile_id bigint NOT NULL,
    intolerance character varying(255)
);

--
-- Name: profile_preferred_cuisines; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_preferred_cuisines (
    profile_id bigint NOT NULL,
    cuisine character varying(255)
);

--
-- Name: profile_preferred_proteins; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.profile_preferred_proteins (
    profile_id bigint NOT NULL,
    protein_source character varying(255)
);

--
-- Name: reminder; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.reminder (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
```

```
        updated_at timestamp(6) with time zone,
        days_of_week character varying(255),
        enabled boolean NOT NULL,
        reminder_time time(6) without time zone,
        title character varying(255) NOT NULL,
        type character varying(255) NOT NULL,
        user_id bigint NOT NULL
    );

--
-- Name: reminder_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.reminder ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.reminder_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: sleep_log; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.sleep_log (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    ended_at timestamp(6) without time zone,
    notes character varying(1000),
    quality integer,
    started_at timestamp(6) without time zone,
    user_id bigint NOT NULL
);

--
-- Name: sleep_log_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.sleep_log ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.sleep_log_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: user_profile; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.user_profile (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    activity_level character varying(255),
    age integer,
    calorie_target integer,
    carb_target integer,
    dietary_preferences character varying(1000),
    disliked_ingredients character varying(1000),
    fat_target integer,
    fitness_goal character varying(255),
    gender character varying(255),
    height_cm double precision,
    hydration_target_ml integer,
    protein_target integer,
    weight_kg double precision,
    user_id bigint NOT NULL,
    budget_preference character varying(30),
    custom_dietary_pattern character varying(200),
    dietary_pattern character varying(40),
    meal_prep_difficulty character varying(30),
    meal_prep_time character varying(30),
    preferred_meals_per_day integer
);

--
-- Name: user_profile_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.user_profile ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.user_profile_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
```

```
-- Name: water_log; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.water_log (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    amount_ml integer NOT NULL,
    logged_at timestamp(6) without time zone NOT NULL,
    user_id bigint NOT NULL
);

--
-- Name: water_log_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.water_log ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.water_log_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: workout; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.workout (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    duration_minutes integer,
    name character varying(255) NOT NULL,
    notes character varying(2000),
    started_at timestamp(6) without time zone,
    user_id bigint NOT NULL
);

--
-- Name: workout_exercise; Type: TABLE; Schema: public; Owner: -
--

CREATE TABLE public.workout_exercise (
    id bigint NOT NULL,
    created_at timestamp(6) with time zone NOT NULL,
    updated_at timestamp(6) with time zone,
    duration_seconds integer,
    notes character varying(1000),
    reps integer,
    sets integer,
    weight_kg double precision,
    exercise_id bigint NOT NULL,
    workout_id bigint NOT NULL,
    exercise_order integer
);

--
-- Name: workout_exercise_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.workout_exercise ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.workout_exercise_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: workout_id_seq; Type: SEQUENCE; Schema: public; Owner: -
--

ALTER TABLE public.workout ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY (
    SEQUENCE NAME public.workout_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1
);

--
-- Name: app_users app_users_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.app_users
    ADD CONSTRAINT app_users_pkey PRIMARY KEY (id);

--
-- Name: body_measurement body_measurement_pkey; Type: CONSTRAINT; Schema: public; Owner: -
```

```
--

ALTER TABLE ONLY public.body_measurement
    ADD CONSTRAINT body_measurement_pkey PRIMARY KEY (id);

--
-- Name: exercise exercise_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.exercise
    ADD CONSTRAINT exercise_pkey PRIMARY KEY (id);

--
-- Name: favourite_exercise favourite_exercise_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.favourite_exercise
    ADD CONSTRAINT favourite_exercise_pkey PRIMARY KEY (id);

--
-- Name: food food_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.food
    ADD CONSTRAINT food_pkey PRIMARY KEY (id);

--
-- Name: generated_meal_plan generated_meal_plan_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.generated_meal_plan
    ADD CONSTRAINT generated_meal_plan_pkey PRIMARY KEY (id);

--
-- Name: generated_workout_plan generated_workout_plan_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.generated_workout_plan
    ADD CONSTRAINT generated_workout_plan_pkey PRIMARY KEY (id);

--
-- Name: goals goals_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.goals
    ADD CONSTRAINT goals_pkey PRIMARY KEY (id);

--
-- Name: grocery_item grocery_item_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.grocery_item
    ADD CONSTRAINT grocery_item_pkey PRIMARY KEY (id);

--
-- Name: grocery_lists grocery_lists_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.grocery_lists
    ADD CONSTRAINT grocery_lists_pkey PRIMARY KEY (id);

--
-- Name: meal_food meal_food_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.meal_food
    ADD CONSTRAINT meal_food_pkey PRIMARY KEY (id);

--
-- Name: meal meal_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.meal
    ADD CONSTRAINT meal_pkey PRIMARY KEY (id);

--
-- Name: notification notification_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.notification
    ADD CONSTRAINT notification_pkey PRIMARY KEY (id);

--
-- Name: reminder reminder_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.reminder
    ADD CONSTRAINT reminder_pkey PRIMARY KEY (id);
```



```
--
-- Name: sleep_log sleep_log_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.sleep_log
    ADD CONSTRAINT sleep_log_pkey PRIMARY KEY (id);

--
-- Name: food uk1ror028moynn6re628ydp848; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.food
    ADD CONSTRAINT uk1ror028moynn6re628ydp848 UNIQUE (barcode);

--
-- Name: app_users uk4vj92ux8a2eehds1mdvmks473; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.app_users
    ADD CONSTRAINT uk4vj92ux8a2eehds1mdvmks473 UNIQUE (email);

--
-- Name: user_profile ukebc21hy5j7scdvcjt0jy6xxrv; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.user_profile
    ADD CONSTRAINT ukebc21hy5j7scdvcjt0jy6xxrv UNIQUE (user_id);

--
-- Name: favourite_exercise ukeyxyerghrvfngyasp3d6vhg9; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.favourite_exercise
    ADD CONSTRAINT ukeyxyerghrvfngyasp3d6vhg9 UNIQUE (user_id, exercise_id);

--
-- Name: exercise ukr1ox34byx8pj01qd39plfobsj; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.exercise
    ADD CONSTRAINT ukr1ox34byx8pj01qd39plfobsj UNIQUE (name);

--
-- Name: user_profile user_profile_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.user_profile
    ADD CONSTRAINT user_profile_pkey PRIMARY KEY (id);

--
-- Name: water_log water_log_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.water_log
    ADD CONSTRAINT water_log_pkey PRIMARY KEY (id);

--
-- Name: workout_exercise workout_exercise_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.workout_exercise
    ADD CONSTRAINT workout_exercise_pkey PRIMARY KEY (id);

--
-- Name: workout workout_pkey; Type: CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.workout
    ADD CONSTRAINT workout_pkey PRIMARY KEY (id);

--
-- Name: idx_goal_user_status; Type: INDEX; Schema: public; Owner: -
--

CREATE INDEX idx_goal_user_status ON public.goals USING btree (user_id, status);

--
-- Name: idx_goal_user_target_date; Type: INDEX; Schema: public; Owner: -
--

CREATE INDEX idx_goal_user_target_date ON public.goals USING btree (user_id, target_date);

--
-- Name: favourite_exercise fk1ltsmrxm2395xhf24wj5nvhe; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.favourite_exercise
    ADD CONSTRAINT fk1ltsmrxm2395xhf24wj5nvhe FOREIGN KEY (exercise_id) REFERENCES public.exercise(id);
```

```
--
-- Name: profile_favourite_foods fk35ohi92mop0ufvnp8jlaghy3b; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.profile_favourite_foods
    ADD CONSTRAINT fk35ohi92mop0ufvnp8jlaghy3b FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: generated_meal_plan fk498sn520aayjt54rrqgsjya44; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.generated_meal_plan
    ADD CONSTRAINT fk498sn520aayjt54rrqgsjya44 FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: water_log fk4ejq0p0iynoichgcwkyjk3sre; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.water_log
    ADD CONSTRAINT fk4ejq0p0iynoichgcwkyjk3sre FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: food fk53o0bv2ep31n35u4gajjbm9l9; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.food
    ADD CONSTRAINT fk53o0bv2ep31n35u4gajjbm9l9 FOREIGN KEY (owner_id) REFERENCES public.app_users(id);

--
-- Name: profile_custom_exclusions fk65n100we2jj2whb51iq4mh5is; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.profile_custom_exclusions
    ADD CONSTRAINT fk65n100we2jj2whb51iq4mh5is FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: profile_preferred_proteins fk7ufeq29xtvs0vgamnr38493j; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.profile_preferred_proteins
    ADD CONSTRAINT fk7ufeq29xtvs0vgamnr38493j FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: body_measurement fk8ceiblxlvaxoyiq8vm14o9c0; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.body_measurement
    ADD CONSTRAINT fk8ceiblxlvaxoyiq8vm14o9c0 FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: meal fk8ll3fjh9lhxngr0ft0ivqj6f; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.meal
    ADD CONSTRAINT fk8ll3fjh9lhxngr0ft0ivqj6f FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: profile_custom_proteins fk97o8gugv7sxvh282u6k5d32dw; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.profile_custom_proteins
    ADD CONSTRAINT fk97o8gugv7sxvh282u6k5d32dw FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: workout fk9dfm98unw7r4v9lo0a9m86l22; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.workout
    ADD CONSTRAINT fk9dfm98unw7r4v9lo0a9m86l22 FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: meal_food fk9eo5x0xh1wkof5eybyeo8h16i; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.meal_food
    ADD CONSTRAINT fk9eo5x0xh1wkof5eybyeo8h16i FOREIGN KEY (meal_id) REFERENCES public.meal(id);

--
-- Name: workout_exercise fkalytxvdcpsg2e2oo8ihk55dm2; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.workout_exercise
    ADD CONSTRAINT fkalytxvdcpsg2e2oo8ihk55dm2 FOREIGN KEY (exercise_id) REFERENCES public.exercise(id);

--
-- Name: generated_workout_plan fkayjihprmlsg0tigrb839rbr2u; Type: FK CONSTRAINT; Schema: public; Owner: -
```

```
--

ALTER TABLE ONLY public.generated_workout_plan
    ADD CONSTRAINT fkayjihprmlsg0tigrb839rbr2u FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: profile_custom_intolerances fkbextg3bhc5y82mxsy6epx321d; Type: FK CONSTRAINT; Schema: public;
-- Owner: -
--

ALTER TABLE ONLY public.profile_custom_intolerances
    ADD CONSTRAINT fkbextg3bhc5y82mxsy6epx321d FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: profile_disliked_foods fkbp7dp7ehsdnngpdklor7kdo3s; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.profile_disliked_foods
    ADD CONSTRAINT fkbp7dp7ehsdnngpdklor7kdo3s FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: favourite_exercise fkd2spc6887mirwraymj6kcgxys; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.favourite_exercise
    ADD CONSTRAINT fkd2spc6887mirwraymj6kcgxys FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: profile_food_intolerances fkd6iqgobxe7mu33kb9d67uh8j0; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.profile_food_intolerances
    ADD CONSTRAINT fkd6iqgobxe7mu33kb9d67uh8j0 FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: grocery_lists fkf497l2am6dany1pjcigxp6m1p; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.grocery_lists
    ADD CONSTRAINT fkf497l2am6dany1pjcigxp6m1p FOREIGN KEY (meal_plan_id) REFERENCES
        public.generated_meal_plan(id);

--
-- Name: profile_cultural_preferences fkfcyoqr10ye76ge83bn9cc7a9; Type: FK CONSTRAINT; Schema: public;
-- Owner: -
--

ALTER TABLE ONLY public.profile_cultural_preferences
    ADD CONSTRAINT fkfcyoqr10ye76ge83bn9cc7a9 FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: meal_food fkgra778wnc9jcsrtYu2ywbny38; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.meal_food
    ADD CONSTRAINT fkgra778wnc9jcsrtYu2ywbny38 FOREIGN KEY (food_id) REFERENCES public.food(id);

--
-- Name: grocery_lists fkgtkfwimv7qdt0sv73asm67wmp; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.grocery_lists
    ADD CONSTRAINT fkgtkfwimv7qdt0sv73asm67wmp FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: sleep_log fkioe5gtawmxbrt6ixjmx5xjq; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.sleep_log
    ADD CONSTRAINT fkioe5gtawmxbrt6ixjmx5xjq FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: profile_dietary_restrictions fkjjxute3hf62cmdo3u9bs7evua; Type: FK CONSTRAINT; Schema: public;
-- Owner: -
--

ALTER TABLE ONLY public.profile_dietary_restrictions
    ADD CONSTRAINT fkjjxute3hf62cmdo3u9bs7evua FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: goals fkksl5wj9u3nmvv9cy7e3bu1nw; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.goals
    ADD CONSTRAINT fkksl5wj9u3nmvv9cy7e3bu1nw FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
```

```
-- Name: profile_custom_allergies fkktm0g9gnhenbbqdx8vgu6rytn; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.profile_custom_allergies
    ADD CONSTRAINT fkktm0g9gnhenbbqdx8vgu6rytn FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: grocery_item fklj59kixv1dpsscvc0u9fv26w3m; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.grocery_item
    ADD CONSTRAINT fklj59kixv1dpsscvc0u9fv26w3m FOREIGN KEY (grocery_list_id) REFERENCES
        public.grocery_lists(id);

--
-- Name: user_profile fko147a12pvkvo2092u5yp6ufv; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.user_profile
    ADD CONSTRAINT fko147a12pvkvo2092u5yp6ufv FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: profile_food_allergies fkoli9hgxji79ct5wns5yllb1s8; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.profile_food_allergies
    ADD CONSTRAINT fkoli9hgxji79ct5wns5yllb1s8 FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: profile_custom_cultural_preferences fkosaqte3ovvctrfma6t23x543x; Type: FK CONSTRAINT; Schema:
    public; Owner: -
--

ALTER TABLE ONLY public.profile_custom_cultural_preferences
    ADD CONSTRAINT fkosaqte3ovvctrfma6t23x543x FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- Name: notification fkqbdwvce7wteewtihpp7ear8w9; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.notification
    ADD CONSTRAINT fkqbdwvce7wteewtihpp7ear8w9 FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: reminder fkqsdemjg334x7x6hsovf3cacy7; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.reminder
    ADD CONSTRAINT fkqsdemjg334x7x6hsovf3cacy7 FOREIGN KEY (user_id) REFERENCES public.app_users(id);

--
-- Name: workout_exercise fkqultuq4g6w47iqdaf0vb8ff3j; Type: FK CONSTRAINT; Schema: public; Owner: -
--

ALTER TABLE ONLY public.workout_exercise
    ADD CONSTRAINT fkqultuq4g6w47iqdaf0vb8ff3j FOREIGN KEY (workout_id) REFERENCES public.workout(id);

--
-- Name: profile_preferred_cuisines fkqxe2l1w1f39ues11wdmay9kgo; Type: FK CONSTRAINT; Schema: public; Owner:
    -
--

ALTER TABLE ONLY public.profile_preferred_cuisines
    ADD CONSTRAINT fkqxe2l1w1f39ues11wdmay9kgo FOREIGN KEY (profile_id) REFERENCES public.user_profile(id);

--
-- PostgreSQL database dump complete
--
```